Schema

While OrientDb can work in a schema-less mode, you may find it necessary at times to enforce a schema on your data model. OrientDB supports both schema-full and schema-hybrid solutions.

In the case of schema-hybrid mode, you only set constraints for certain fields and leave the user to add custom fields to the record. This mode occurs at a class level, meaning that you can have an `Employee` class as schema-full and an `EmployeeInformation` class as schema-less.

- **Schema-full** Enables strict-mode at a class-level and sets all fields as mandatory.
- **Schema-less** Enables classes with no properties. Default is non-strict-mode, meaning that records can have arbitrary fields.
- **Schema-hybrid** Enables classes with some fields, but allows records to define custom fields. This is also sometimes called schema-mixed.

> **NOTE** Changes to the schema are not transactional. You must execute these commands outside of a transaction.

You can access the schema through SQL or through the Java API. Examples here use the latter. To access the schema API in Java, you need the Schema instance of the database you want to use. For example,

```
OSchema schema = database.getMetadata().getSchema();
```

Class

OrientDB draws from the Object Oriented programming paradigm in the concept of the Class. A class is a type of record. In comparison to Relational database systems, it is most similar in conception to the table.

Classes can be schema-less, schema-full or schema-hybrid. They can inherit from other classes, shaping a tree of classes. In other words, a sub-class extends the parent class, inheriting all attributes.

Each class has its own clusters. By default, these clusters are logical, but they can also be physical. A given class must have at least one cluster defined as its default, but it can support multiple clusters. OrientDB writes new records into the default cluster, but always reads from all defined clusters.

When you create a new class, OrientDB creates a default physical cluster that uses the same name as the class, but in lowercase.

Creating Persistent Classes

Classes contain one or more properties. This mode is similar to the classical model of the Relational database, where you must define tables before you can begin to store records.

To create a persistent class in Java, use the `createClass()` method:

```
OClass account = database.getMetadata().getSchema().createClass("Account");
```

This method creates the class `Account` on the database. It simultaneously creates the physical cluster `account`, to provide storage for records in the class `Account`.

Getting Persistent Classes

With the new persistent class created, you may also need to get its contents.

To retrieve a persistent class in Java, use the `getClass()` method:

```
OClass account = database.getMetadata().getSchema().getClass("Account");
```

This method retrieves from the database the persistent class `Account`. If the query finds that the `Account` class does not exist, it returns `NULL`.

Dropping Persistent Classes

In the event that you no longer want the class, you can drop, or delete, it from the database.

To drop a persistent class in Java, use the `OSchema.dropClass()` method:

```
database.getMetadata().getSchema().dropClass("Account");
```

This method drops the class `Account` from your database. It does not delete records that belong to this class unless you explicitly ask it to do so:

```
database.command(new OCommandSQL("DELETE FROM Account")).execute();
database.getMetadata().getSchema().dropClass("Account");
```

Constraints

Working in schema-full mode requires that you set the strict mode at the class-level, by defining the `setStrictMode()` method to `TRUE`. In this case, records of that class cannot have undefined properties.

Properties

In OrientDB, a property is a field assigned to a class. For the purposes of this tutorial, consider Property and Field as synonymous.

Creating Class Properties

After you create a class, you can define fields for that class. To define a field, use the `createProperty()` method.

```
OClass account = database.getMetadata().getSchema().createClass("Account");
account.createProperty("id", OType.Integer);
account.createProperty("birthDate", OType.Date);
```

These lines create a class `Account`, then defines two properties `id` and `birthDate`. Bear in mind that each field must belong to one of the supported types. Here these are the integer and date types.

Dropping Class Properties

In the event that you would like to remove properties from a class you can do so using the `dropProperty()` method under `OClass`.

```
database.getMetadata().getSchema().getClass("Account").dropProperty("name");
```

When you drop a property from a class, it does not remove records from that class unless you explicitly ask for it, using the `UPDATE... REMOVE` statements. For instance,

```
database.getMetadata().getSchema().getClass("Account").dropProperty("name");
database.command(new OCommandSQL("UPDATE Account REMOVE name")).execute();
```

The first method drops the property from the class. The second updates the database to remove the property.

Relationships

OrientDB supports two types of relationships: referenced and embedded.

Referenced Relationships

In the case of referenced relationships, OrientDB uses a direct link to the referenced record or records. This allows the database to avoid the costly `JOIN` operations used by Relational databases.

```
                  customer
  Record A     ------------->    Record B
CLASS=Invoice                 CLASS=Customer
  RID=5:23                       RID=10:2
```

In the example, Record A contains the reference to Record B in the property `customer`. Both records are accessible by any other records since each has a Record ID.

1:1 and *n*:1 Reference Relationships

In one to one and many to one relationships, the reference relationship is expressed using the `LINK` type. For instance.

```
OClass customer= database.getMetadata().getSchema().createClass("Customer");
customer.createProperty("name", OType.STRING);

OClass invoice = database.getMetadata().getSchema().createClass("Invoice");
invoice.createProperty("id", OType.INTEGER);
invoice.createProperty("date", OType.DATE);
invoice.createProperty("customer", OType.LINK, customer);
```

Here, records of the class `Invoice` link to a record of the class `Customer`, through the field `customer`.

1:*n* and *n*:*n* Reference Relationships.

In one to many and many to many relationships, OrientDB expresses the referenced relationship using collections of links.

- `LINKLIST` An ordered list of links.
- `LINKSET` An unordered set of links, that does not accept duplicates.
- `LINKMAP` An ordered map of links, with a string key. It does not accept duplicate keys.

For example,

```
OClass orderItem = db.getMetadata().getSchema().createClass("OrderItem");
orderItem.createProperty("id", OType.INTEGER);
orderItem.createProperty("animal", OType.LINK, animal);

OClass order = db.getMetadata().getSchema().createClass("Order");
order.createProperty("id", OType.INTEGER);
order.createProperty("date", OType.DATE);
order.createProperty("items", OType.LINKLIST, orderItem);
```

Here, you have two classes: `Order` and `OrderItem` and a 1:*n* referenced relationship is created between them.

Embedded Relationships

In the case of embedded relationships, OrientDB contains the relationship within the record. Embedded relationships are stronger than referenced relationships, but the embedded record does not have its own Record ID. Because of this, you cannot reference them directly through other records. The relationship is only accessible through the container record. If the container record is deleted, then the embedded record is also deleted.

```
                  address
  Record A     <>---------->   Record B
CLASS=Account               CLASS=Address
  RID=5:23                     NO RID!
```

Here, Record A contains the entirety of Record B in the property `address`. You can only reach Record B by traversing the container, Record A.

```
orientdb>
SELECT FROM Account WHERE Address.city = 'Rome'
```

1:1 and *n*:1 Embedded Relationships

For one to one and many to one embedded relationships, OrientDB uses links of the `EMBEDDED` type. For example,

```
OClass address = database.getMetadata().getSchema().createClass("Address");

OClass account = database.getMetadata().getSchema().createClass("Account");
account.createProperty("id", OType.INTEGER);
account.createProperty("birthDate", OType.DATE);
account.createProperty("address", OType.EMBEDDED, address);
```

Here, records of the class `Account` embed records for the class `Address`.

1:*n* and *n*:*n* Embedded Relationships

In the case of one to many and many to many relationships, OrientDB sues a collection embedded link types:

- `EMBEDDEDLIST` An ordered list of records.
- `EMBEDDEDSET` An unordered set of records. It doesn't accept duplicates.
- `EMBEDDEDMAP` An ordered map of records as key-value pairs. It doesn't accept duplicate keys.

For example,

```
OClass orderItem = db.getMetadata().getSchema().createClass("OrderItem");
orderItem.createProperty("id", OType.INTEGER);
orderItem.createProperty("animal", OType.LINK, animal);

OClass order = db.getMetadata().getSchema().createClass("Order");
order.createProperty("id", OType.INTEGER);
order.createProperty("date", OType.DATE);
order.createProperty("items", OType.EMBEDDEDLIST, orderItem);
```

This establishes a one to many relationship between the classes `Order` and `OrderItem`.

Constraints

OrientDB supports a number of constraints for each field. For more information on setting constraints, see the `ALTER PROPERTY` command.

- **Minimum Value**: `setMin()` The field accepts a string, because it works also for date ranges.
- **Maximum Value**: `setMax()` The field accepts a string, because it works also for date rangers.
- **Mandatory**: `setMandatory()` This field is required.
- **Read Only**: `setReadonly()` This field cannot update after being created.
- **Not Null**: `setNotNull()` This field cannot be null.

- **Unique**: This field doesn't allow duplicates or speedup searches.
- **Regex**: This field must satisfy Regular Expressions

For example,

```
profile.createProperty("nick",
OType.STRING).setMin("3").setMax("30").setMandatory(true).setNotNull(true);
profile.createIndex("nickIdx", OClass.INDEX_TYPE.UNIQUE, "nick"); // Creates
unique constraint

profile.createProperty("name", OType.STRING).setMin("3").setMax("30");
profile.createProperty("surname", OType.STRING).setMin("3").setMax("30");
profile.createProperty("registeredOn", OType.DATE).setMin("2010-01-01
00:00:00");
profile.createProperty("lastAccessOn", OType.DATE).setMin("2010-01-01
00:00:00");
```

Indices as Constraints

To define a property value as unique, use the `UNIQUE` index constraint. For example,

```
profile.createIndex("EmployeeId", OClass.INDEX_TYPE.UNIQUE, "id");
```

You can also constrain a group of properties as unique by creating a composite index made from multiple fields. For instance,

```
profile.createIndex("compositeIdx", OClass.INDEX_TYPE.NOTUNIQUE, "name",
"surname");
```

For more information about indexes look at Index guide.